Projekt 2 – manager pakietów

Termin oddania: 3/4 listopada 2026.

Opis problemu

Manager pakietów (np. `apt`, `pip`, `conda`) instaluje programy razem z ich zależnościami. **Repozytorium** zawiera pakiety, a każdy pakiet ma jedną lub więcej **wersji**. Każda wersja może:

- **wymagać** innych pakietów, np. `lib >=2.0 <3.0` (dowolna wersja `lib` co najmniej 2.0 i mniejsza niż 3.0); wymaganie może być **alternatywą**, np. `json-fast | json-slow` (wystarczy jeden z tych pakietów);
- **konfliktować** z innymi pakietami, np. `conflicts util >=2.0` (nie może być zainstalowana razem z `util` w wersji 2.0 lub nowszej).

W środowisku może być zainstalowana co najwyżej jedna wersja każdego pakietu. **Stan środowiska** to zbiór zainstalowanych par (pakiet, wersja). Stan jest **spójny**, jeśli każda zainstalowana wersja ma spełnione wszystkie wymagania i nie konfliktuje z żadnym zainstalowanym pakietem. Jak w `apt`, rozróżniamy pakiety zainstalowane **ręcznie** (na życzenie użytkownika) i **automatycznie** (jako zależności).

Celem projektu jest zbudowanie bazy wiedzy o repozytorium w logice zdań i odpowiadanie na zapytania managera pakietów (instalacja, usunięcie, aktualizacja, wyjaśnienie, dlaczego czegoś nie da się zainstalować) za pomocą solvera SAT/SMT. Zalecany jest Python i solver Z3 (`pip install z3-solver`). Można też użyć innego solvera z obsługą optymalizacji, np. PySAT (MaxSAT) albo clingo (ASP). Można też napisać własny algorytmu przeszukiwania, jednak jest to trudniejsza opcja i nie jest wymagane.

Zapytania mają zwykle wiele rozwiązań, więc wybieramy najlepsze według kilku kryteriów w ustalonej kolejności (optymalizacja **leksykograficzna**: najpierw minimalizujemy pierwsze kryterium, wśród rozwiązań optymalnych dla niego drugie itd.). W kryteriach występuje **przestarzałość** wersji, czyli jej pozycja na liście wersji pakietu posortowanej od najnowszej (0 = najnowsza). Przestarzałość zbioru pakietów to suma przestarzałości ich zainstalowanych wersji.

Formaty danych

Plik repozytorium jest plikiem JSON. Obiekt `packages` przypisuje nazwie pakietu obiekt z polem `versions`, a ten – każdej wersji obiekt z opcjonalnymi listami `requires` (zależności) i `conflicts` (konflikty). Opcjonalne pole `meta` na najwyższym poziomie (np. opis repozytorium) jest ignorowane. Inne pola są niedozwolone:

```
{
  "meta": {"description": "Fragment repozytorium przykładowego"},
  "packages": {
```

```
    "app":    {"versions": {"2.0": {"requires": ["lib >=2.0", "util"]}}},
    "lib":    {"versions": {"2.0": {"requires": ["util >=2.0"]}}},
    "util":   {"versions": {"1.0": {}, "2.0": {}}},
    "tool":   {"versions": {"1.1": {"requires": ["lib <2.0", "json-fast | json-slow >=1.0"]}
    "legacy": {"versions": {"1.0": {"conflicts": ["util >=2.0"]}}}
  }
}
```

- **Nazwa pakietu** składa się z liter, cyfr i znaków `-`, `_`, `.`. Wielkość liter nie ma znaczenia, a ciągi znaków `-`, `_`, `.` są równoważne (jak w PyPI), więc `Foo_Bar`, `foo-bar` i `foo.bar` to ten sam pakiet.
- **Wersja** to liczby całkowite oddzielone kropkami, porównywane jako ciągi liczb, więc `1.10 > 1.9`, a `1.0` i `1.0.0` to ta sama wersja. Kolejność wersji w pliku jest dowolna.
- **Wymaganie** to nazwa pakietu, po której występuje zero lub więcej ograniczeń `OP WERSJA` (operatory `==`, `!=`, `<`, `<=`, `>`, `>=`), oddzielonych spacjami lub przecinkami. Wszystkie ograniczenia muszą być spełnione jednocześnie. Spacja między operatorem a wersją jest opcjonalna, więc `lib==2.0` i `lib == 2.0` są równoważne.
- Element listy `requires` jest napisem z jednym wymaganiem albo kilkoma wymaganiami oddzielonymi `|` (alternatywa). Element listy `conflicts` zawiera jedno wymaganie. Pakiet nie może wymagać samego siebie ani konfliktować z samym sobą.
- Wymaganie, któremu nie odpowiada żadna wersja w repozytorium, jest dozwolone (należy wypisać ostrzeżenie). Wersja z takim wymaganiem jest po prostu nieinstalowalna.

Plik stanu zawiera po jednym pakiecie w linii: `NAZWA WERSJA`, a dla pakietów zainstalowanych ręcznie `NAZWA WERSJA manual`.

Żądanie użytkownika ma tę samą postać co wymaganie, np. `app`, `lib<2.0`, `lib >=1.0 <2.0`, `app==2.0`.

Plik żądań jest plikiem tekstowym z jednym żądaniem w linii. Linie puste są pomijane, a znak `#` rozpoczyna komentarz do końca linii. Program przyjmuje zbiór żądań wyłącznie z takiego pliku – w argumentach wywołania podaje się tylko ścieżkę do niego, np. `python manager.py install zadania.txt --state stan.txt`. Niepoprawne żądanie należy zgłosić z numerem linii. W ten sam sposób (jedna nazwa pakietu w linii) podaje się zbiór pakietów P do usunięcia (zadanie 6.2) i do aktualizacji (zadanie 8.1); dla aktualizacji plik jest opcjonalny (brak pliku = wszystkie pakiety).

```
# zadania.txt - instalacja app w wersji 2.0 razem z tool
app==2.0
tool
```

Do projektu dołączono pliki `repo_example.json` (repozytorium przykładowe), `state_example.txt` (przykładowy stan) i `requests_example.txt` (przykładowy plik żądań). Użyj ich do sprawdzenia swojej implementacji.

Repozytoria do eksperymentów. Każda grupa wybiera numer z pliku "numery grup.xlsx" odpowiadający numerowi jednego z członków grupy.otrzymuje od prowadzącego numer $k \in \{1, \ldots, 40\}$ i w zadaniach 4.5, 6.3 i 6.4 pracuje **wyłącznie** z repozytoriami ze swojego katalogu `repos/kXX/` (np. `repos/k07/` dla $k = 7$). Katalog zawiera repozytoria `repo_N.json` dla $N \in \{10, 30, 100, 300, 1000\}$ pakietów, pliki żądań `requests_N.txt` do zadania 4.5 oraz repozytorium `repo_200.json` do zadania 6.4. Repozytoria powstały z danych PyPI (dla dużych N także z „forków" pakietów PyPI pod zmienionymi nazwami) i zawierają pakiety ilustrujące trudne sytuacje – pliki mogą generować ostrzeżenia, a niektóre żądania mogą nie mieć rozwiązania. W raporcie podaj swój numer k. Dla instalacji do pustego środowiska (kryteria z zadania 4.3):

żądanie	wynik
`app`	`app 2.0`, `lib 2.0`, `util 2.0`
`tool`	`tool 1.1`, `lib 1.0`, `json-slow 1.0`
`app tool`	`app 1.0`, `tool 1.1`, `lib 1.0`, `json-slow 1.0`
`app legacy`	`app 1.0`, `legacy 1.0`, `lib 1.0`
`app==2.0 tool`	brak rozwiązania

Dla stanu z `state_example.txt` (kryteria z zadań 6.1–6.2 i 8.1):

zapytanie	zmiany
install `app==2.0`	`app` 1.0 → 2.0, `lib` 1.0 → 2.0, instalacja `util 2.0`, usunięcie `tool`
remove `lib`	usunięcie `lib`, `app`, `tool`
remove `tool` z autoremove	usunięcie `tool`, `json-slow`
update (wszystko)	`lib` 1.0 → 1.5, instalacja `util 2.0`

Zadania

Progi punktowe są kumulatywne: na 6 pkt trzeba wykonać także zadania na 4 pkt, a na 8 pkt – zadania na 4 i 6 pkt.

Zadania na 4 punkty (instalacja do pustego środowiska)

1. Napisz program wczytujący repozytorium z pliku w podanym formacie. Repozytorium, wymagania i stan reprezentuj za pomocą wyodrębnionych klas. Program ma odrzucać niepoprawne pliki z czytelnym komunikatem wskazującym miejsce błędu: dla błędów składni JSON numer linii, a dla pozostałych ścieżkę w dokumencie, np. `packages['app'].versions['2.0'].requires[1]`. Przykłady błędów: niepoprawny JSON, brak pola `packages`, nieznane pole, wartość złego typu (np. `requires` nie jest listą napisów), niepoprawna nazwa, wersja

lub operator, powtórzona wersja (także `1.0` i `1.0.0`), powtórzony pakiet (także `Foo_Bar` i `foo-bar`), pakiet wymagający samego siebie. Uwaga: standardowy `json.loads` po cichu pomija powtórzone klucze obiektu – pomocny jest parametr `object_pairs_hook`.

2. Zbuduj bazę wiedzy w logice zdań. Wprowadź zmienną `p==v` („zainstalowano pakiet p w wersji v") dla każdej wersji w repozytorium i zapisz aksjomaty:
   - (A1) co najwyżej jedna wersja każdego pakietu,
   - (A2) zależności: `p==v` pociąga za sobą spełnienie każdego wymagania (alternatywy wymagań) wersji v,
   - (A3) konflikty: `p==v` wyklucza każdą pasującą wersję pakietu, z którym konfliktuje.

   Każdy aksjomat ma mieć czytelną etykietę, np. `app==2.0 wymaga lib >=2.0`, bo przyda się w zadaniu 6.3. W raporcie opisz kodowanie i podaj formuły dla kilku wersji z `repo_example.json`.
3. Zaimplementuj instalację zbioru żądań R do pustego środowiska. Wynik musi być stanem spójnym, w którym spełnione są wszystkie żądania. Kryteria (leksykograficznie):
   1. minimalna przestarzałość pakietów żądanych (użytkownik chce możliwie nowych wersji tego, o co prosi),
   2. minimalna liczba zainstalowanych pakietów,
   3. minimalna przestarzałość pozostałych pakietów.

   Jeśli rozwiązanie nie istnieje, program ma to czytelnie zgłosić.
4. Napisz procedurę weryfikującą stan **bez użycia solvera**. Procedura ma sprawdzić, czy każda wersja istnieje w repozytorium, czy wszystkie wymagania są spełnione, czy nie ma konfliktów i (opcjonalnie) czy spełnione są podane żądania. Ma też wypisać **wszystkie** naruszone warunki. Weryfikuj nią każdy wynik zwrócony przez solver.
5. Przeprowadź eksperyment na repozytoriach swojej grupy: dla każdego $N \in \{10, 30, 100, 300, 1000\}$ zainstaluj w pustym środowisku zbiór żądań z pliku `repos/kXX/requests_N.txt` w repozytorium `repos/kXX/repo_N.json`. W raporcie podaj dla każdego problemu: liczbę wersji i aksjomatów, żądanie, wynik (liczba zainstalowanych pakietów i ich przestarzałość albo brak rozwiązania), czas budowy bazy wiedzy (z wczytaniem pliku), czas rozwiązania oraz wynik weryfikacji. Skomentuj, jak czasy zależą od rozmiaru repozytorium.

Zadania na 6 punktów (stan środowiska, usuwanie, wyjaśnianie)

1. Rozszerz instalację o istniejący stan środowiska (wczytywany z pliku i zapisywany do pliku). Pakiety żądane stają się pakietami ręcznymi. Kryteria (leksykograficznie):
   1. minimalna liczba usuniętych pakietów,
   2. minimalna liczba zmian wersji pakietów, które pozostają zainstalowane,
   3. minimalna przestarzałość pakietów żądanych, które nie były zain-

stalowane,
   4. minimalna liczba nowych pakietów,
   5. minimalna przestarzałość pozostałych nowych pakietów.

   Dla pustego stanu kryteria te sprowadzają się do kryteriów z zadania 4.3. Program ma wypisywać plan zmian (instalacja, usunięcie, podniesienie lub obniżenie wersji).
2. Zaimplementuj usunięcie zbioru pakietów P. Żaden pakiet z P nie może pozostać zainstalowany, a pakiety od nich zależne trzeba usunąć albo zmienić ich wersję. Kryteria:
   1. minimalna liczba usuniętych pakietów ręcznych spoza P,
   2. minimalna liczba zmian wersji,
   3. minimalna liczba nowych pakietów,
   4. minimalna liczba usuniętych pakietów.

   Dodaj opcję *autoremove*. Wtedy w kryterium 4 minimalizujemy liczbę **wszystkich** zainstalowanych pakietów, czyli usuwamy także zależności, które przestały być potrzebne.
3. Zaimplementuj wyjaśnianie, dlaczego zbioru żądań nie da się zainstalować. Program ma wypisać **minimalny** sprzeczny podzbiór aksjomatów i żądań (*unsat core*): po usunięciu dowolnego jego elementu reszta przestaje być sprzeczna. Etykiety mają być czytelne dla człowieka. Pokaż wyjaśnienia dla `app==2.0 tool` z repozytorium przykładowego i dla przypadków bez rozwiązania z zadania 4.5 (jeśli wystąpiły; jeśli nie – skonstruuj kilka).
4. Przeprowadź scenariusz co najmniej 10 losowych operacji (instalacja, usunięcie z autoremove i bez niego) na repozytorium `repos/kXX/repo_200.json` swojej grupy ($N = 200$), zaczynając od pustego stanu. Ustal i podaj ziarno generatora liczb losowych, aby scenariusz dało się odtworzyć. Każdy stan pośredni zweryfikuj procedurą z zadania 4.4. W raporcie podaj dla każdej operacji: liczbę zmian, liczbę pakietów (w tym ręcznych) i czas.

Zadania na 8 punktów (aktualizacja, dane z PyPI)

1. Zaimplementuj aktualizację zbioru zainstalowanych pakietów P (domyślnie wszystkich). Pakiety ręczne i pakiety z P muszą pozostać zainstalowane. Kryteria:
   1. minimalna przestarzałość pakietów z P,
   2. minimalna liczba usuniętych pakietów,
   3. minimalna liczba zmian wersji pakietów spoza P,
   4. minimalna liczba nowych pakietów,
   5. minimalna przestarzałość wszystkich pakietów.
2. Aktualizacja nie zawsze ma jedno najlepsze rozwiązanie. W `repo_example.json` najnowsze `a` wymaga starego `b`, a najnowsze `b` – starego `a`. Suma przestarzałości jest jednym z możliwych kompromisów, ale nie jedynym. Rozszerz aktualizację tak, aby:
   - dla każdego pakietu z P wyznaczyć najnowszą wersję osiągalną osobno (przy zachowaniu pakietów ręcznych),
   - jeśli tych wersji nie da się zainstalować jednocześnie, zgłosić kon-

flikt, podać jego przyczynę (unsat core z zadania 6.3) oraz wypisać wszystkie rozwiązania nieporównywalne, czyli front Pareto względem przestarzałości pakietów w konflikcie.

   Dla stanu `a 1.0 manual`, `b 1.0 manual` frontem są `{a 2.0, b 1.0}` i `{a 1.1, b 2.0}`. Aktualizacja samego `a` daje `a 2.0` bez konfliktu.

3. Wczytaj bazę wiedzy z pliku `pypi_data.json` (92 pakiety, 3330 wersji z PyPI, ze zbioru najpopularniejszych pakietów i ich zależności). Plik ma postać `{"meta": {"environment": {...}}, "packages": {"requests": {"versions": {"2.32.3": {"requires_dist": ["idna<4,>=2.5", ...], "requires_python": ">=3.8"}, ...}}, ...}}`. Wersje i wymagania mają tu format PEP 440 / PEP 508 i należy je interpretować biblioteką `packaging`. Pomiń wersje niezgodne ze środowiskiem docelowym (`requires_python`). Znaczniki środowiska (`; sys_platform == "win32"`) wartościuj dla środowiska zapisanego w `meta.environment`. Dodatki (*extras*) pomiń. Reszta programu (zapytania, weryfikacja, wyjaśnianie) ma działać bez zmian. Przeprowadź i opisz w raporcie scenariusz, np.: instalacja najstarszych dostępnych `pandas` i `requests`; doinstalowanie `boto3`, a potem `aiobotocore` (wymaga wąskiego zakresu wersji `botocore`); aktualizacja `requests`; aktualizacja wszystkiego; usunięcie `botocore`; wyjaśnienie, dlaczego nie da się zainstalować najnowszego `boto3` z najstarszym `botocore`. Podaj czasy wczytania i rozwiązania.

Pytanie dodatkowe (nieobowiązkowe): pokaż, że problem „czy żądanie R da się zainstalować?" jest NP-zupełny. Wskazówka: każdą formułę 3-CNF da się zakodować jako repozytorium. Jakie elementy formatu (alternatywy, konflikty, ograniczenia wersji) są do tego potrzebne? Czy problem pozostaje trudny bez konfliktów i bez alternatyw?

Wskazówki

Liczba wszystkich stanów środowiska to $\prod_p (k_p + 1)$, gdzie k_p jest liczbą wersji pakietu p (każdy pakiet jest niezainstalowany albo zainstalowany w jednej z wersji). Już dla 30 pakietów po 3 wersje daje to $4^{30} \approx 10^{18}$ stanów. Przeszukanie wyczerpujące jest więc realne tylko dla bardzo małych repozytoriów, ale **warto** go użyć w testach: dla repozytoriów z kilkoma pakietami można porównać wynik solvera z minimum znalezionym przez przejrzenie wszystkich stanów.

Podstawowe elementy Z3 w Pythonie:

```
import z3

x = {("lib", "1.0"): z3.Bool("lib==1.0"), ("lib", "2.0"): z3.Bool("lib==2.0"), ...}
opt = z3.Optimize()
opt.set(priority="lex")                                  # cele leksykograficzne
opt.add(z3.AtMost(x["lib", "1.0"], x["lib", "2.0"], 1))   # (A1)
opt.add(z3.Implies(x["app", "2.0"], z3.Or(x["lib", "2.0"])))   # (A2)
```

```
opt.minimize(z3.Sum([z3.If(x[k], 1, 0) for k in x]))      # kolejne wywołania = kolejne cel
if opt.check() == z3.sat:
    m = opt.model()
    print([k for k in x if z3.is_true(m.eval(x[k]))])
```

- Do wyjaśnień służy `z3.Solver` z `s.set("core.minimize", True)`, `s.assert_and_track(formuła, z3.Bool("etykieta"))` i `s.unsat_core()`.
- Front Pareto daje `opt.set(priority="pareto")`: każde kolejne wywołanie `opt.check()` zwraca następne rozwiązanie z frontu.
- W danych z PyPI przydają się `packaging.version.Version` (porządek wersji), `packaging.requirements.Requirement` (parsowanie `requires_dist`), `SpecifierSet.contains(v, prereleases=True)` i `Marker.evaluate({**env, "extra": ""})`.
- Pusta alternatywa (`z3.Or()` bez argumentów) jest fałszem. To naturalnie koduje wymaganie, którego nie spełnia żadna wersja w repozytorium.

Co oddać

- kod źródłowy z instrukcją uruchomienia;
- przykładowe pliki wejściowe (repozytoria, stany, pliki żądań);
- raport z wynikami i komentarzem do zadań z wybranego progu;
- projekt należy zaprezentować prowadzącemu w trakcie zajęć i odpowiedzieć na ewentualne pytania;
- *(zalecane)* testy automatyczne, np. sprawdzające, że każdy wynik przechodzi weryfikację i że na małych repozytoriach wynik solvera jest optymalny (porównanie z przeszukiwaniem wyczerpującym).